

DevOps Engineer Assessment

CI/CD Automation for .NET Application Deployment to Minikube

Documentation and Presentation

Prepared for: Significa

Repository: github.com/Ge0rak/minikube-cicd

1. Overview

This document describes the implementation of a complete CI/CD pipeline that builds, tests, containerizes, and deploys a minimal ASP.NET Core Web API to a local Kubernetes (Minikube) cluster, using Azure DevOps Pipelines end to end. It covers all five core tasks of the assessment brief plus all five bonus tasks, and closes with the challenges encountered and the key insights gained.

The public repository, containing the full commit history referenced throughout this document, is available at:

<https://github.com/Ge0rak/minikube-cicd>

2. Architecture

The final architecture combines two separate Azure Pipelines, a public GitHub mirror for pre-merge validation, and a self-hosted agent to bridge the gap between cloud-hosted CI and a locally-running Kubernetes cluster.

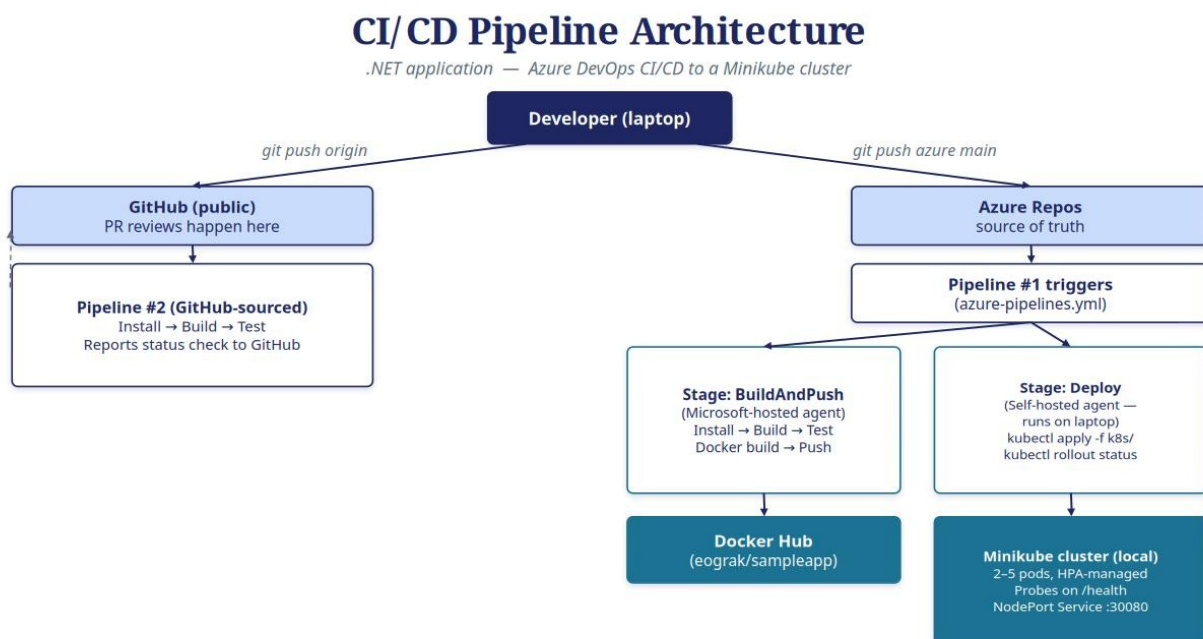


Figure 1: CICD Pipeline Architecture

Prometheus and Grafana run in-cluster (via Helm) in parallel, scraping CPU/Memory metrics from every pod and visualizing them on demand.

3. Task 1 — Azure DevOps Configuration

An Azure DevOps organization and project (minikube-cicd) were created, with a Git repository hosted in Azure Repos as the project's source of truth. Because this Azure DevOps organization

only offered private project visibility at creation time, the repository was additionally mirrored to a public GitHub repository to satisfy the assessment's requirement for a public repository with visible commit history.

A minimal ASP.NET Core 10 Web API (SampleApp) was scaffolded, exposing two endpoints:

- GET /weatherforecast — the default sample data endpoint from the .NET Web API template
- GET /health — a custom endpoint added specifically for later use in Kubernetes probes

A YAML-based build pipeline (azure-pipelines.yml) was configured to trigger automatically on every commit to main, installing the .NET SDK, restoring dependencies, and building the application.

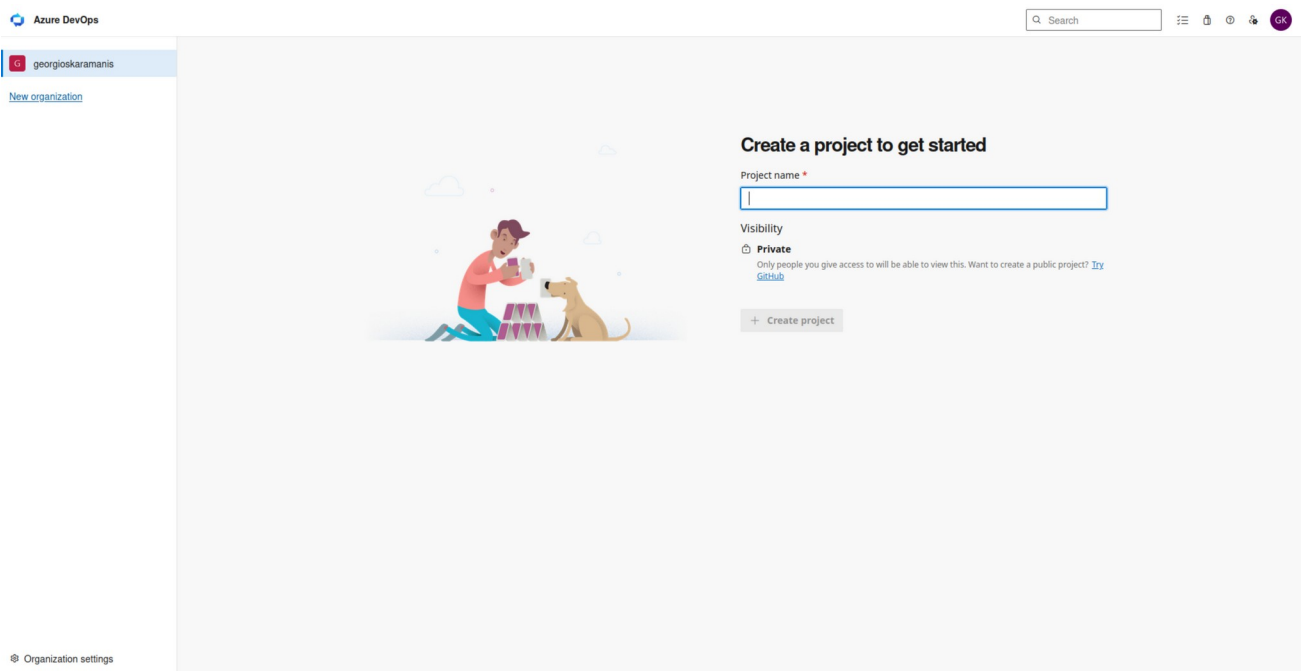


Figure 2: Azure - Create new project

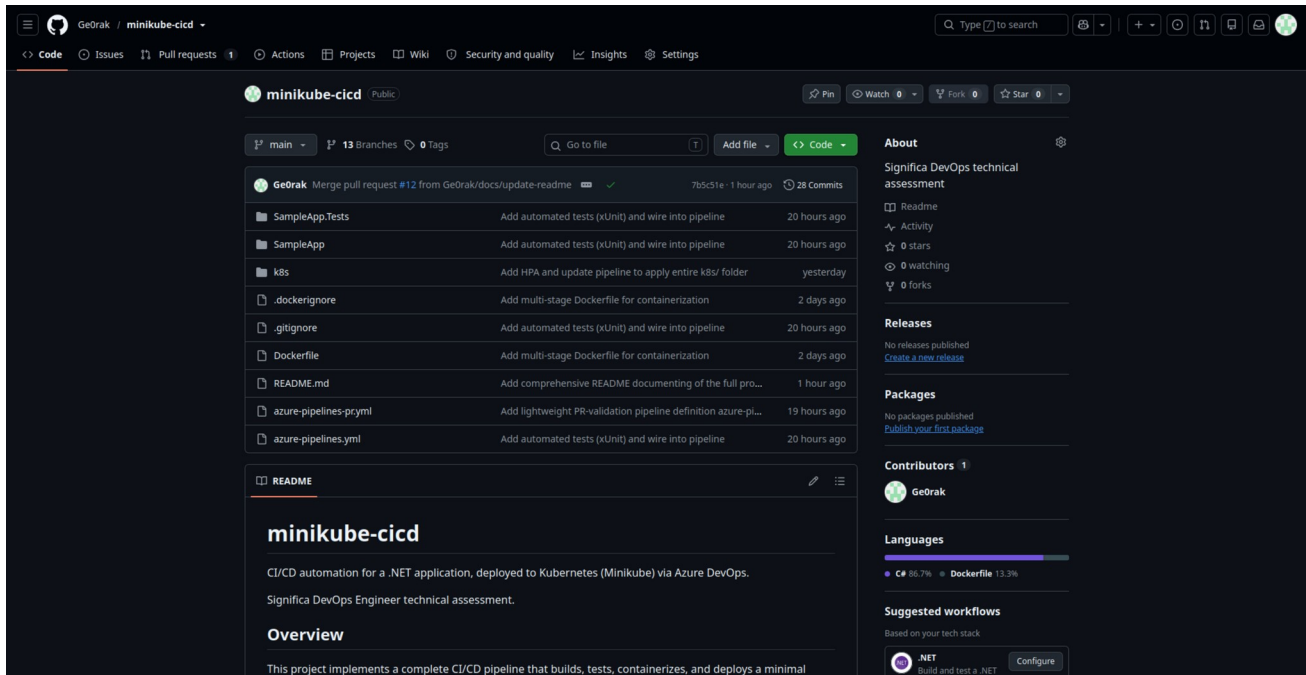


Figure 3: GitHub repo – Remote Origin

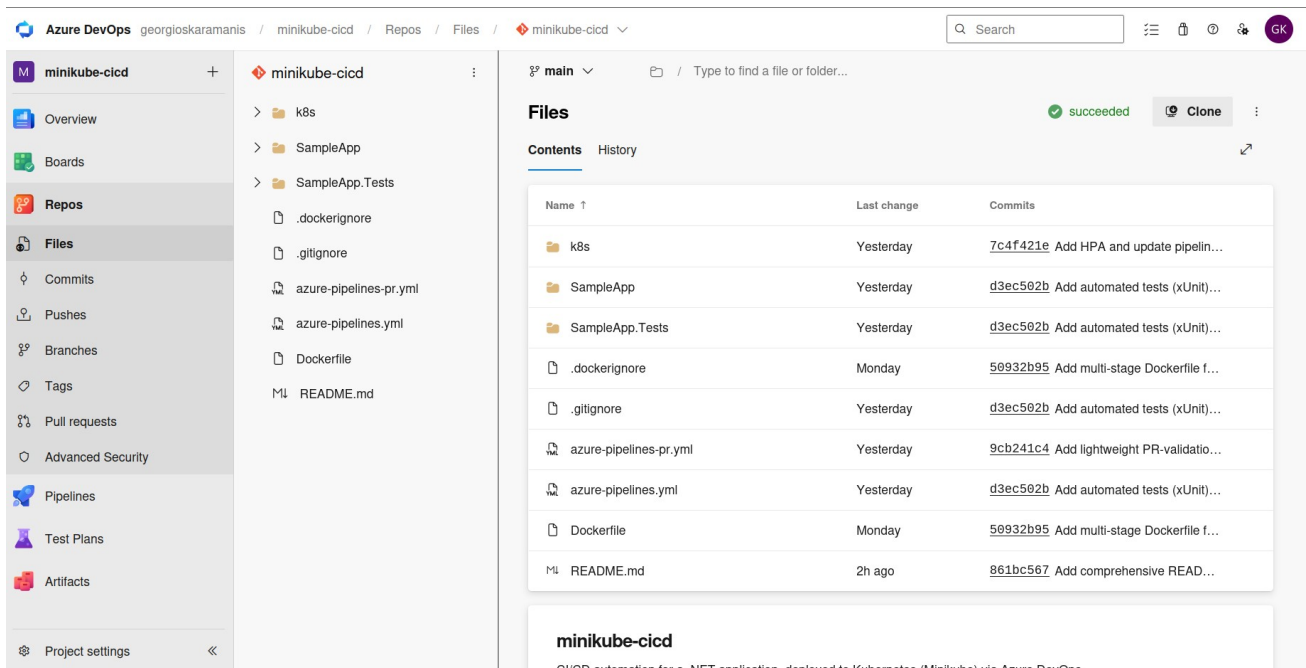


Figure 4: Azure repo - Remote azure (mirror)

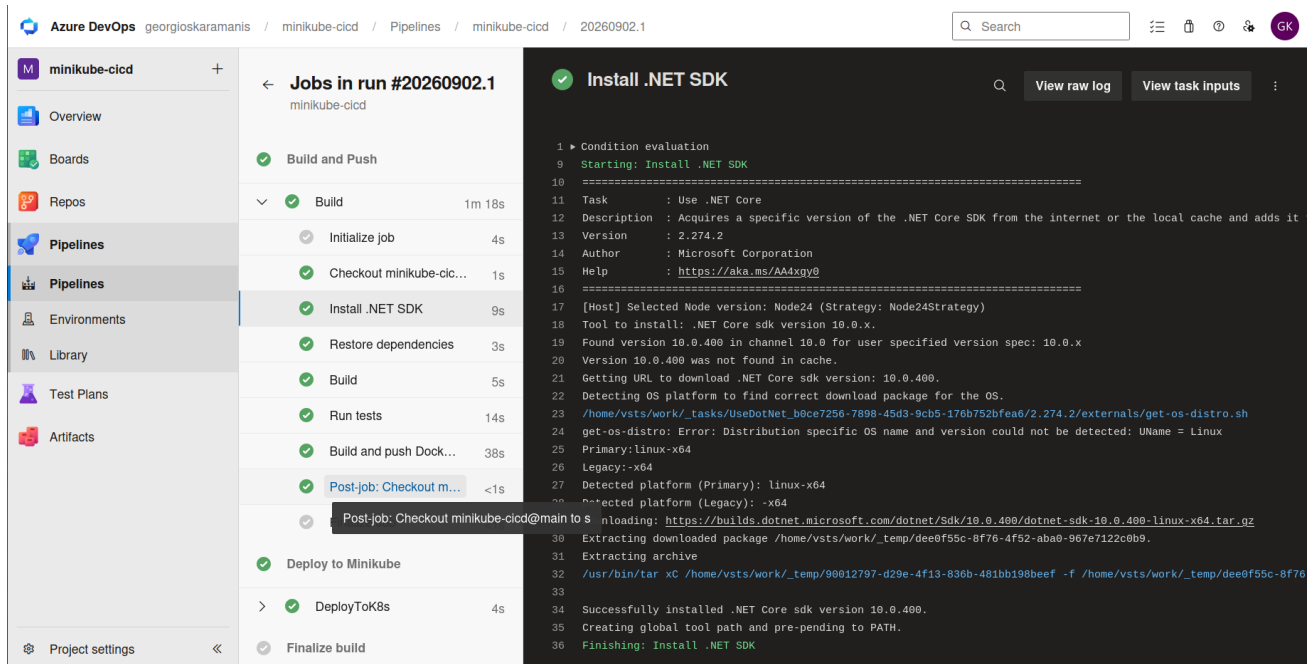


Figure 5: Azure pipeline - .NET SDK installation

Key decisions

- YAML pipelines were used instead of the classic UI-based pipeline editor, so the pipeline definition itself is version-controlled alongside the application code.
- A feature-branch and Pull Request workflow was adopted from the start and kept consistent throughout the entire project, giving a clean, incremental commit history.

4. Task 2 — Docker Image Creation

Docker Engine (the official Docker CE distribution, rather than the snap or apt-bundled alternatives) was installed for better compatibility with Minikube. A multi-stage Dockerfile was written:

- Build stage — uses the .NET 10 SDK image to restore, build, and publish the application
- Runtime stage — uses the much smaller ASP.NET runtime image, copying in only the published output from the build stage

The application listens on port 8080 inside the container. A `.dockerignore` file excludes local `bin/` and `obj/` build artifacts from the Docker build context — this was necessary to fix a real issue encountered during development, described in the Challenges section.

The pipeline was extended with a Docker build-and-push task that tags the image with both a unique build ID and the latest tag, then pushes it to Docker Hub using a securely configured service connection.

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
```

```

COPY SampleApp/SampleApp.csproj SampleApp/
RUN dotnet restore SampleApp/SampleApp.csproj
COPY SampleApp/ SampleApp/
WORKDIR /src/SampleApp
RUN dotnet publish -c Release -o /app/publish --no-restore

```

```

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "SampleApp.dll"]

```

```

eog@eog-ThinkPad-X270-W100G:~/GitHub/minkube-cicd$ docker build -t sampleapp:local .
[+] Building 7.2s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> == transferring dockerfile: 530B
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:10.0
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:10.0
=> [internal] load .dockerignore
=> == transferring context: 70B
=> [build 1/7] FROM mcr.microsoft.com/dotnet/sdk:10.0@sha256:e1ff2a92ae84c1291bc1b6887501f8af98e6331e7af6d4c8d37108c5e87a64c
=> == resolve mcr.microsoft.com/dotnet/sdk:10.0@sha256:e1ff2a92ae84c1291bc1b6887501f8af98e6331e7af6d4c8d37108c5e87a64c
=> [runtime 1/3] FROM mcr.microsoft.com/dotnet/aspnet:10.0@sha256:a4556ed033fa96f904bb7a0d348851cb2d30b1201662420070e45f664fbb5f94
=> == resolve mcr.microsoft.com/dotnet/aspnet:10.0@sha256:a4556ed033fa96f904bb7a0d348851cb2d30b1201662420070e45f664fbb5f94
=> [internal] load build context
=> == transferring context: 363B
=> == CACHED [runtime 2/3] WORKDIR /app
=> == CACHED [build 2/7] WORKDIR /src
=> == CACHED [build 3/7] COPY SampleApp/SampleApp.csproj SampleApp/
=> == CACHED [build 4/7] RUN dotnet restore SampleApp/SampleApp.csproj
=> [build 5/7] COPY SampleApp/ SampleApp/
=> [build 6/7] WORKDIR /src/SampleApp
=> [build 7/7] RUN dotnet publish -c Release -o /app/publish --no-restore
=> [runtime 3/3] COPY --from=build /app/publish .
=> == exporting to image
=> == exporting layers
=> == exporting manifest sha256:92de16bc0e05bc45def19f8f439470c250db070e484fda2705abe7a2bb05ee0
=> == exporting config sha256:9c8e03a3df10f11013db0b0caded1d702a64a4f005b05eb754e73266fee3b6d
=> == exporting attestation manifest sha256:a7cf01f369b4e24ecaf6105d44fe02b27670ab9fe079b2bab00ec9a2020154d
=> == exporting manifest list sha256:a022d619d368860b97a83eebedb71b1ccaa9b4b491da9ff1fbf6e06001f0e203
=> == naming to docker.io/library/sampleapp:local
=> == unpacking to docker.io/library/sampleapp:local
eog@eog-ThinkPad-X270-W100G:~/GitHub/minkube-cicd$ docker images | grep sampleapp
sampleapp:local    a022d619d368    341MB    96.2MB
eog@eog-ThinkPad-X270-W100G:~/GitHub/minkube-cicd$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
eog@eog-ThinkPad-X270-W100G:~/GitHub/minkube-cicd$ docker run -d -p 8080:8080 --name sampleapp-test sampleapp:local
d3d320f7c38b12465f6a839a57fa1a26416/did0c525022bb06d27df3905148
eog@eog-ThinkPad-X270-W100G:~/GitHub/minkube-cicd$ curl http://localhost:8080/health
{"date":"2026-09-01","temperature":24,"summary":"Bracing","temperature":175},{"date":"2026-09-02","temperature":130,"summary":"Scorching","temperature":100},{"date":"2026-09-03","temperature":54,"summary":"Chilly","temperature":129},{"date":"2026-09-04","temperature":11,"summary":"Chilly","temperature":151},{"date":"2026-09-05","temperature":22,"summary":"Bracing","temperature":77]}eog@eog-ThinkPad-X270-W100G:~/GitHub/minkube-cicd$ curl http://localhost:8080/health
{"status":"healthy"}eog@eog-ThinkPad-X270-W100G:~/GitHub/minkube-cicd$ docker stop sampleapp-test
docker rm sampleapp-test

```

Figure 6: Docker sample app build locally for testing

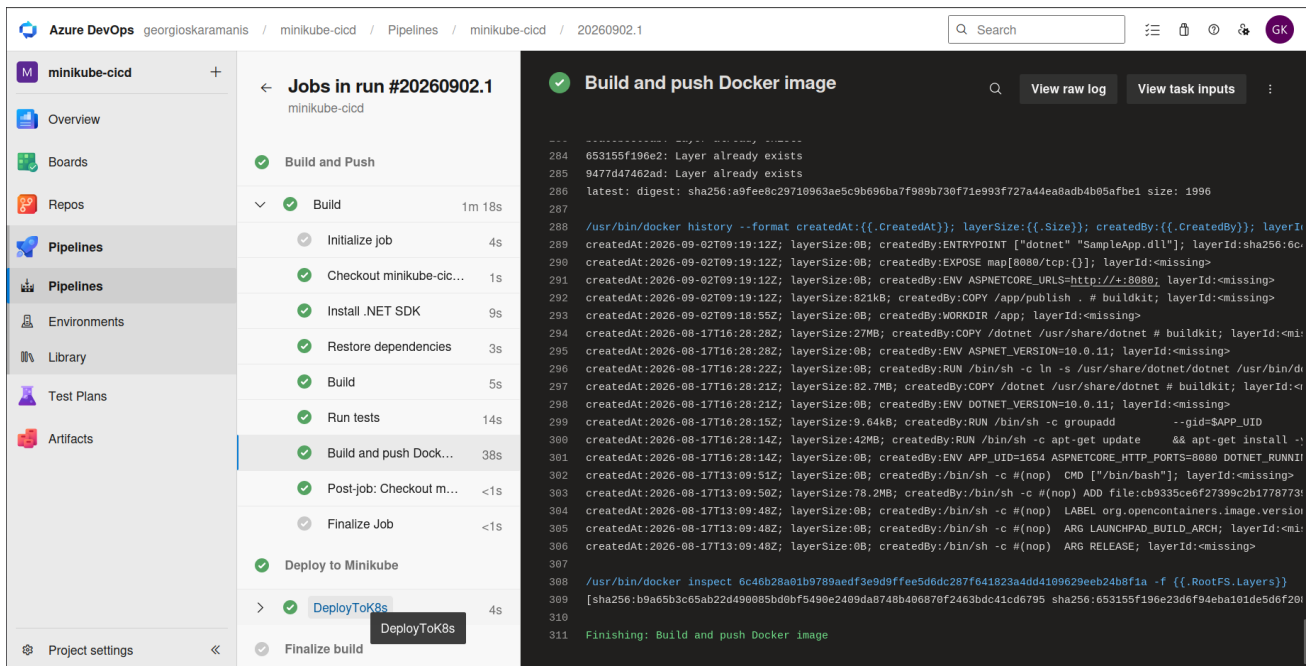


Figure 7: Azure pipeline - Build Docker image and push to Dockerhub

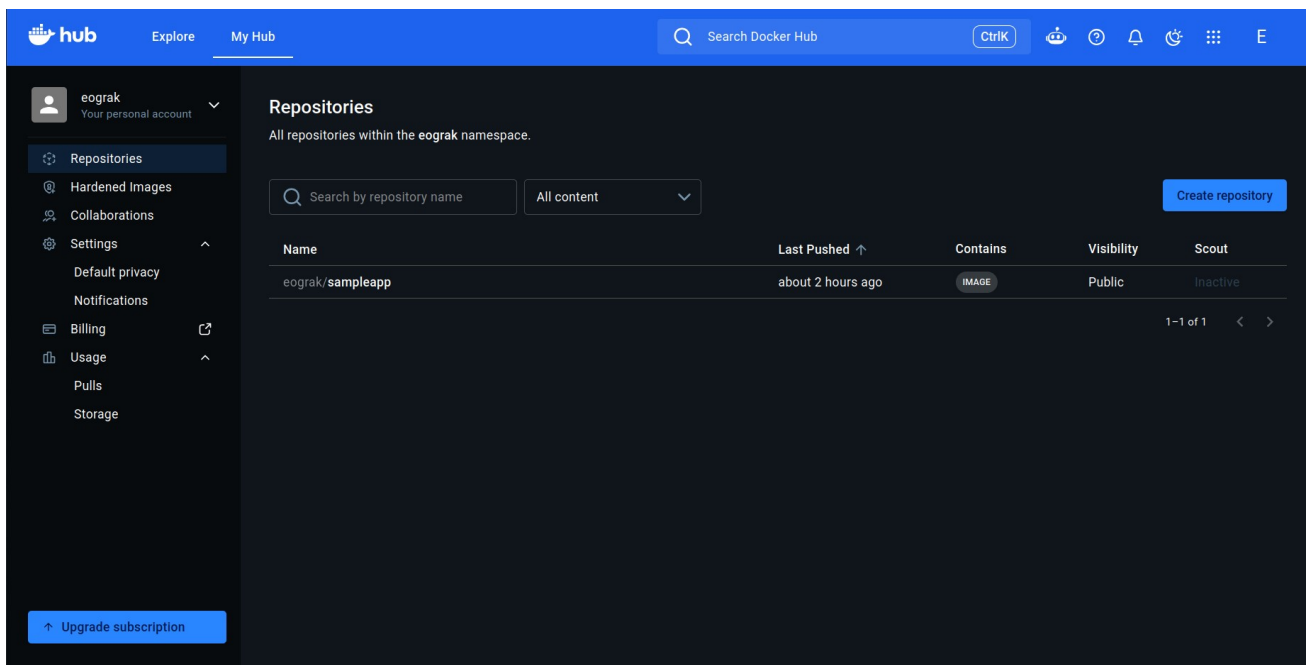


Figure 8: DockerHub repo for sample app docker image

Key decisions

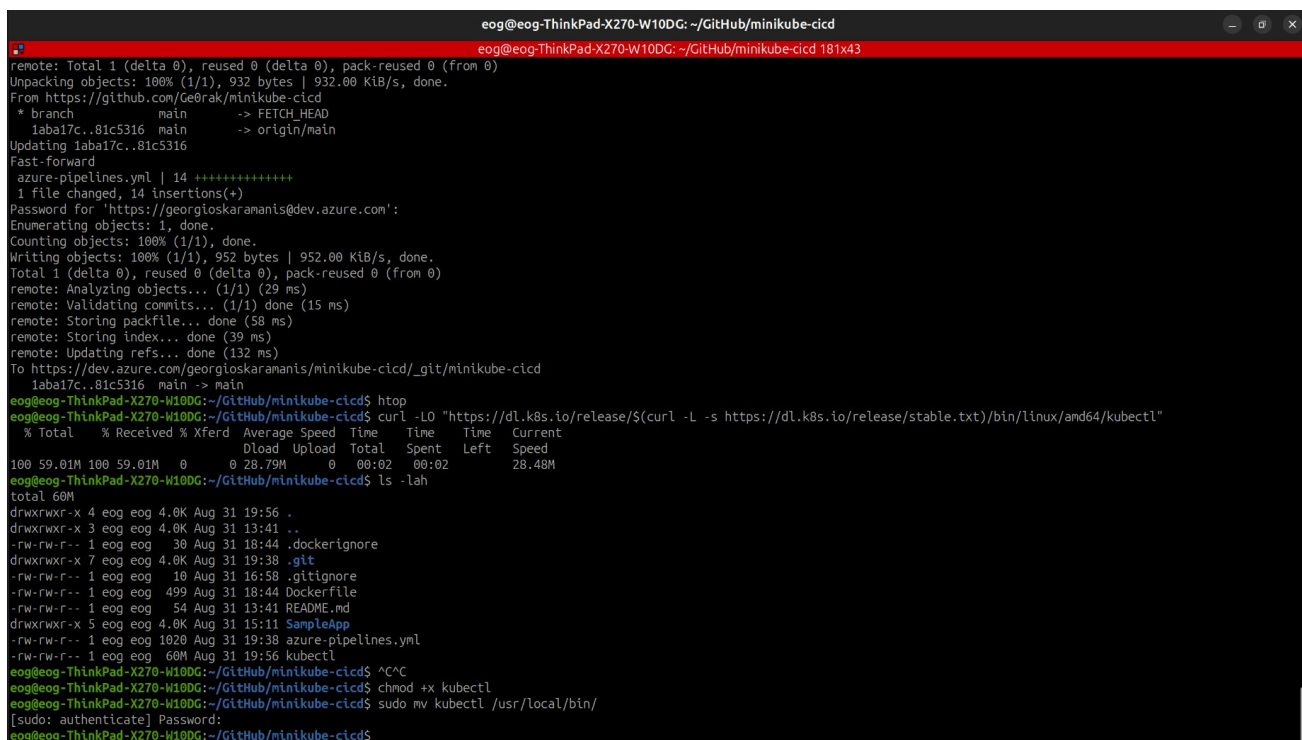
- Docker Hub was chosen over Azure Container Registry, since it is genuinely free for public repositories, while ACR's cheapest tier carries a small ongoing cost.

- A multi-stage build was chosen over building outside Docker, producing a smaller final image and following the industry-standard, self-contained pattern for containerized .NET applications.

5. Task 3 — Kubernetes Deployment

kubectl and Minikube were installed via their official binaries, and a single-node Kubernetes cluster was started locally using the Docker driver. Kubernetes manifests were written for a Deployment (2 replicas, pulling the application image from Docker Hub) and a NodePort Service exposing the application on port 30080.

The most significant architectural challenge in this task was that Azure's Microsoft-hosted build agents run on cloud virtual machines with no network route to a laptop's local Minikube cluster. This was solved by installing a self-hosted Azure Pipelines agent directly on the laptop, registered to the project's Default agent pool. The pipeline was then restructured into two stages: BuildAndPush, unchanged, running on the Microsoft-hosted agent; and a new Deploy stage, running on the self-hosted agent, which executes kubectl apply against the local cluster and waits for the rollout to complete.



```
eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 932 bytes | 932.00 KiB/s, done.
From https://github.com/Ge0rak/minikube-cicd
 * branch      main      -> FETCH_HEAD
    1ab17c..81c5316  main      -> origin/main
Updating 1ab17c..81c5316
Fast-forward
 azure-pipelines.yml | 14 ++++++
 1 file changed, 14 insertions(+)
Password for 'https://georgioskaramanis@dev.azure.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 952 bytes | 952.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Analyzing objects... (1/1) (29 ms)
remote: Validating commits... (1/1) done (15 ms)
remote: Storing packfile... done (58 ms)
remote: Storing index... done (39 ms)
remote: Updating refs... done (132 ms)
To https://dev.azure.com/georgioskaramanis/minikube-cicd/_git/minikube-cicd
    1ab17c..81c5316  main -> main
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ htop
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ curl -LO "https://dl.k8s.io/release/${curl -L -s https://dl.k8s.io/release/stable.txt}/bin/linux/amd64/kubectl"
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload Total   Spent    Left     Speed
100 59.01M 100 59.01M    0     0  28.79M    0  00:02  00:02    28.48M
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ls -lah
total 60M
drwxrwxr-x 4 eog eog 4.0K Aug 31 19:56 .
drwxrwxr-x 3 eog eog 4.0K Aug 31 13:41 ..
-rw-rw-r-- 1 eog eog 30 Aug 31 18:44 .dockerignore
drwxrwxr-x 7 eog eog 4.0K Aug 31 19:38 .git
-rw-rw-r-- 1 eog eog 10 Aug 31 16:58 .gitignore
-rw-rw-r-- 1 eog eog 499 Aug 31 18:44 Dockerfile
-rw-rw-r-- 1 eog eog 54 Aug 31 13:41 README.md
drwxrwxr-x 5 eog eog 4.0K Aug 31 15:11 SampleApp
-rw-rw-r-- 1 eog eog 1020 Aug 31 19:38 azure-pipelines.yml
-rw-rw-r-- 1 eog eog 60M Aug 31 19:56 kubectl
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ^C^C
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ chmod +x kubectl
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ sudo mv kubectl /usr/local/bin/
[sudo: authenticate] Password:
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$
```

Figure 9: Kubectl installation locally


```
eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd
eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd 181x43

Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 344 bytes | 344.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Analyzing objects... (3/3) (26 ms)
remote: Validating commits... (1/1) done (22 ms)
remote: Storing packfile... done (43 ms)
remote: Storing index... done (50 ms)
remote: Updating refs... done (127 ms)
To https://dev.azure.com/georgioskaramanis/minikube-cicd/_git/minikube-cicd
81c5316..cdda4b2 main -> main
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ minikube start --driver=docker
🐳 minikube v1.38.1 on Ubuntu 26.04
🌟 Using the docker driver based on user configuration
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
🔥 Using Docker driver with root privileges
👉 Starting "minikube" primary control-plane node in "minikube" cluster
📡 Pulling base image v0.0.50 ...
📡 Downloading Kubernetes v1.35.1 preload ...
> preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 13.56 M
> gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 16.44 M
🔥 Creating docker container (CPUs=2, Memory=3072MB) ...
📡 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
  ▪ Enabled addons: storage-provisioner, default-storageclass

! /usr/local/bin/kubectl is version 1.37.0, which may have incompatibilities with Kubernetes 1.35.1.
  ▪ Want kubectl v1.35.1? Try 'minikube kubectl -- get pods -A'
👉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get nodes
NAME      STATUS   ROLES    AGE   VERSION
minikube  Ready    control-plane  112s  v1.35.1
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$
```

Figure 10: Start minikube locally

```
eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd
eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd 181x43

spec:
  type: NodePort
  selector:
    app: sampleapp
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30808
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ls -lah
total 40K
drwxrwxr-x 5 eog eog 4.0K Aug 31 20:20 .
drwxrwxr-x 3 eog eog 4.0K Aug 31 13:41 ..
-rw-rw-r-- 1 eog eog 30 Aug 31 18:44 .dockerignore
drwxrwxr-x 7 eog eog 4.0K Aug 31 20:12 .git
-rw-rw-r-- 1 eog eog 60 Aug 31 20:10 .gitignore
-rw-rw-r-- 1 eog eog 499 Aug 31 18:44 Dockerfile
-rw-rw-r-- 1 eog eog 54 Aug 31 13:41 README.md
drwxrwxr-x 5 eog eog 4.0K Aug 31 15:11 SampleApp
-rw-rw-r-- 1 eog eog 1020 Aug 31 19:38 azure-pipelines.yml
drwxrwxr-x 2 eog eog 4.0K Aug 31 20:21 k8s
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ls -lah k8s/
total 16K
drwxrwxr-x 2 eog eog 4.0K Aug 31 20:21 .
drwxrwxr-x 5 eog eog 4.0K Aug 31 20:20 ..
-rw-rw-r-- 1 eog eog 384 Aug 31 20:21 deployment.yaml
-rw-rw-r-- 1 eog eog 188 Aug 31 20:21 service.yaml
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ^C^C
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl apply -f k8s/deployment.yaml
deployment.apps/sampleapp-deployment created
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl apply -f k8s/service.yaml
service/sampleapp-service created
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
sampleapp-deployment-55cf4d675d-mtjgs 1/1     Running   0           33s
sampleapp-deployment-55cf4d675d-wvpm9 1/1     Running   0           33s
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
sampleapp-deployment 2/2      2             2           67s
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get services
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes     ClusterIP   10.96.0.1    <none>         443/TCP           10m
sampleapp-service NodePort    10.103.77.129 <none>        80:30808/TCP     61s
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$
```

Figure 11: k8sn manifests applied locally

```
eog@eog-ThinkPad-X270-W10DG: ~/myagent 181x20
agent v5.278.0 (commit e907351)

>> End User License Agreements:

Building sources from a TFVC repository requires accepting the Team Explorer Everywhere End User License Agreement. This step is not required for building sources from Git repositories.

A copy of the Team Explorer Everywhere license agreement can be found at:
/home/eog/myagent/license.html

Enter (Y/N) Accept the Team Explorer Everywhere license agreement now? (press enter for N) > y

>> Connect:

Enter server URL > https://dev.azure.com/georgioskaramanis
```

Figure 12: Self-hosted Azure Pipelines agent installation

```
eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd
eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd 181x20
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
sampleapp-deployment-55cf4d675d-ntjgs 1/1     Running   0           152m
sampleapp-deployment-55cf4d675d-wvpm9 1/1     Running   0           152m
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl rollout status deployment/sampleapp-deployment
deployment "sampleapp-deployment" successfully rolled out
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl describe deployment sampleapp-deployment | grep Image
Image:
  eograk/sampleapp:latest
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ^C
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ^C
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ curl http://192.168.49.2:30080/health
{"status":"Healthy"}eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ curl http://192.168.49.2:30080/health^C
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ ^C
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ curl http://192.168.49.2:30080/weatherforecast
[{"date":"2026-09-01","temperatureC":44,"summary":"Warm","temperatureF":111}, {"date":"2026-09-02","temperatureC":40,"summary":"Scorching","temperatureF":103}, {"date":"2026-09-03","temperatureC":51,"summary":"Warm","temperatureF":123}, {"date":"2026-09-04","temperatureC": -7,"summary":"Bracing","temperatureF":20}, {"date":"2026-09-05","temperatureC":29,"summary":"Freezing","temperatureF":84}]eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$

eog@eog-ThinkPad-X270-W10DG: ~/myagent 181x20
Enter agent pool (press enter for default) >
Enter agent name (press enter for eog-ThinkPad-X270-W10DG) >
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) >
2026-08-31 18:30:07Z: Settings Saved.
eog@eog-ThinkPad-X270-W10DG:~/myagent$ ./run.sh
Scanning for tool capabilities.
Connecting to the server.
2026-08-31 18:31:22Z: Listening for Jobs
^CExiting...
eog@eog-ThinkPad-X270-W10DG:~/myagent$ ./run.sh
Scanning for tool capabilities.
Connecting to the server.
2026-08-31 19:23:02Z: Listening for Jobs
2026-08-31 19:46:44Z: Running job: DeployToK8s
2026-08-31 19:46:58Z: Job DeployToK8s completed with result: Succeeded
```

Figure 13: Azure local agent active and k8s cluster API test

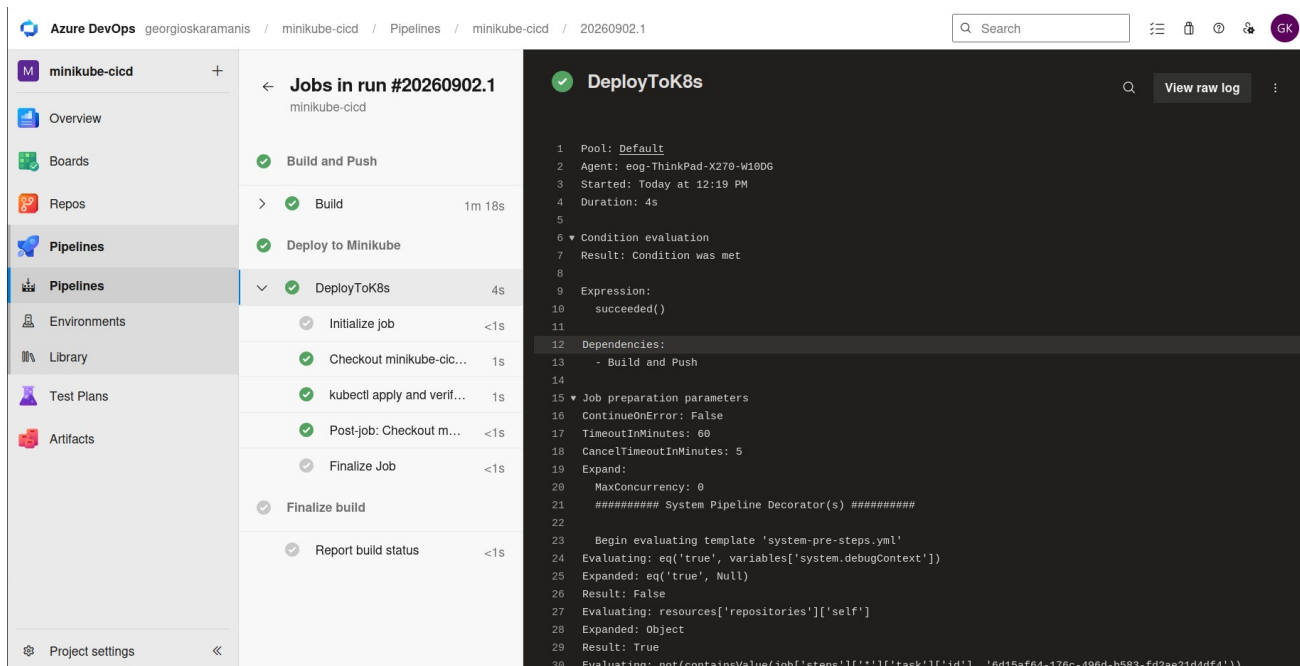


Figure 14: Azure pipeline - Deploy k8s cluster locally via agent

Key decisions

- A self-hosted agent was the only viable way to let a cloud-hosted CI/CD platform reach a cluster that only exists locally — a deliberate, well-understood trade-off: the laptop and the agent process must both be running for automated deploys to trigger, mirroring how real organizations use self-hosted agents to reach internal or on-premises infrastructure.

6. Task 4 — Monitoring

Rather than only describing a monitoring approach, a working monitoring stack was implemented. Helm was installed, and the kube-prometheus-stack chart was deployed into a dedicated monitoring namespace, bringing in Prometheus, Grafana, Alertmanager, and supporting metrics exporters.

Grafana was accessed locally via `kubectl port-forward`, and live metrics were confirmed for the application's pods, including CPU and Memory utilization, requests, and limits. Resource requests and limits were subsequently added to the Deployment — both a Kubernetes best practice and a prerequisite for the Horizontal Pod Autoscaler bonus task — and the change was verified to flow automatically through the pipeline and appear live in Grafana within moments.

```
eeg@eeg-ThinkPad-X270-W10DG: ~  
eeg@eeg-ThinkPad-X270-W10DG: ~ 236x45  
% Total    % Received % Xferd  Average Speed   Time    Time     Current  
          Dload  Upload  Total   Spent    Left     Speed  
100 12252 100 12252    0     0 41133      0  0 0 0 0  
Downloading https://get.helm.sh/helm-v3.21.4-linux-amd64.tar.gz  
Verifying checksum... done.  
Preparing to install helm into /usr/local/bin  
[sudo: authenticate] Password:  
helm installed into /usr/local/bin/helm  
eeg@eeg-ThinkPad-X270-W10DG:~$ helm version  
version.BuildInfo{Version: "v3.21.4", GitCommit: "813176c51bb5c181dbbd7901298ddcc104cd3417", GitTreeState: "clean", GoVersion: "go1.20.5"}  
eeg@eeg-ThinkPad-X270-W10DG:~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts  
"prometheus-community" has been added to your repositories  
eeg@eeg-ThinkPad-X270-W10DG:~$ helm repo update  
Hang tight while we grab the latest from your chart repositories...  
...Successfully got an update from the "prometheus-community" chart repository  
Update Complete. Happy Helming!  
eeg@eeg-ThinkPad-X270-W10DG:~$ kubectl create namespace monitoring  
namespace/monitoring created  
eeg@eeg-ThinkPad-X270-W10DG:~$ helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring  
NAME: prometheus  
LAST DEPLOYED: Tue Sep 11 11:59:03 2020  
NAMESPACE: monitoring  
STATUS: deployed  
REVISION: 1  
TEST SUITE: None  
NOTES:  
kube-prometheus-stack has been installed. Check its status by running:  
  kubectl --namespace monitoring get pods -l "release=prometheus"  
  
Get Grafana 'admin' user password by running:  
  
  kubectl --namespace monitoring get secrets prometheus-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo  
  
Access Grafana local instance:  
  
  export POD_NAME=$(kubectl --namespace monitoring get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=prometheus" -oname)  
  kubectl --namespace monitoring port-forward $POD_NAME 3000  
  
Get your grafana admin user password by running:  
  
  kubectl get secret --namespace monitoring -l app.kubernetes.io/component=admin-secret -o jsonpath="{.items[0].data.admin-password}" | base64 --decode ; echo  
  
Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.  
eeg@eeg-ThinkPad-X270-W10DG:~$  
eeg@eeg-ThinkPad-X270-W10DG: ~ 236x45  
eeg@eeg-ThinkPad-X270-W10DG:~$ cd ~/myagent ./run.sh  
bash: cd: too many arguments  
eeg@eeg-ThinkPad-X270-W10DG:~$ cd ~/myagent  
eeg@eeg-ThinkPad-X270-W10DG:~$ ./myagents ./run.sh  
Scanning for tool capabilities.  
Connecting to the server.  
2020-09-01 00:26:38Z: Listening for Jobs  
]
```

Figure 15: Helm local installation and configuration

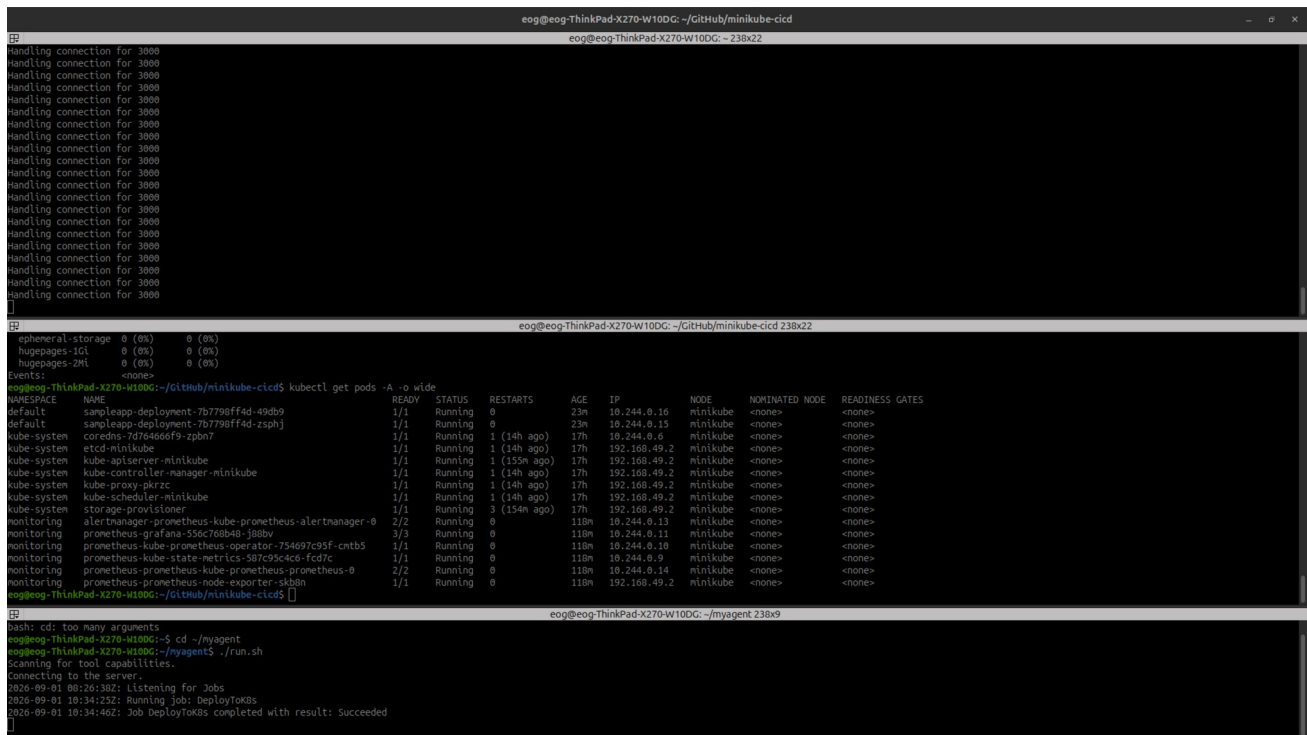


Figure 16: Grafana port forwarding | All pods | Local agent

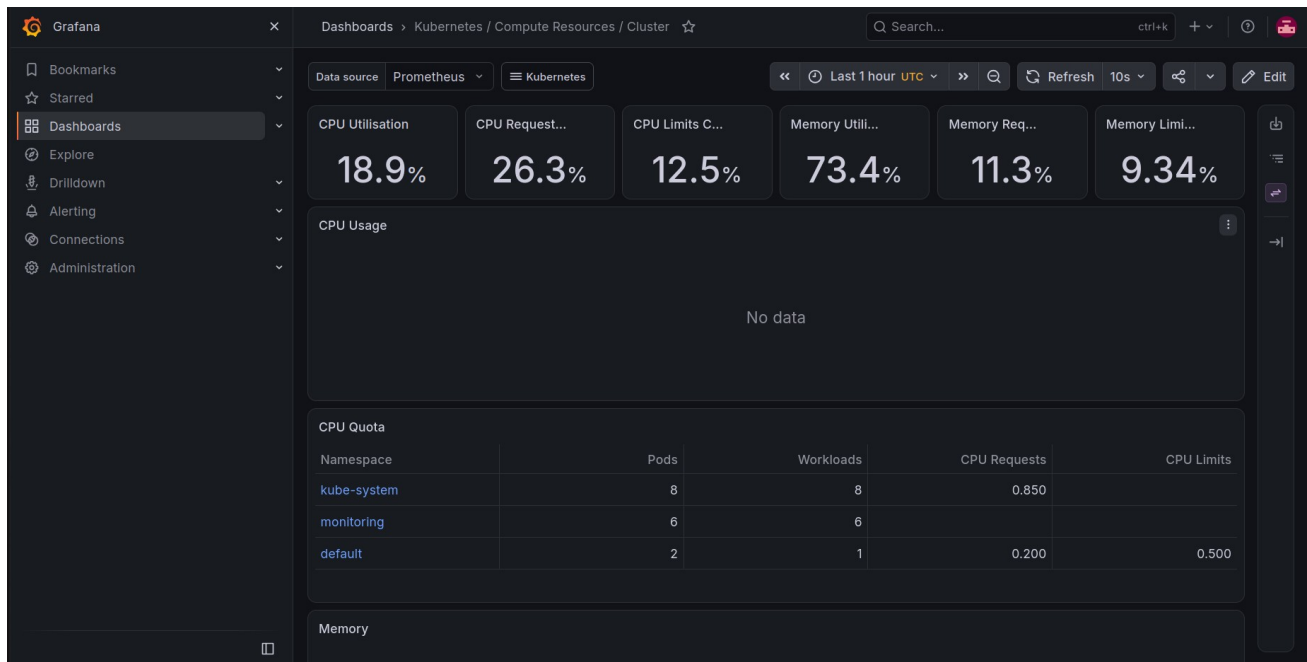


Figure 17: Grafana Dashboard - Cluster resources

KPIs tracked

KPI	Coverage
CPU Saturation	Covered — Prometheus + Grafana, per-pod and per-namespace

Memory Usage	Covered — Prometheus + Grafana
Error Rate (proxy)	Covered — Pod restart count
Availability	Covered — Deployment replica health (N/N ready)
Latency	Known gap — would require application-level instrumentation (e.g. prometheus-net), not implemented in this project

7. Bonus Task — Liveness, Readiness, and Startup Probes

All three Kubernetes probe types were added to the Deployment manifest, each checking the existing GET /health endpoint on port 8080:

- Startup probe — checks every 1 second, allowing up to 30 seconds for the application to start before Kubernetes gives up. Since this application starts in under a second, this demonstrates handling a slow-starting application gracefully rather than solving a real problem for this specific app.
- Liveness probe — checks every 10 seconds, restarting the container only after 3 consecutive failures (30 seconds), avoiding restarts triggered by a single transient blip.
- Readiness probe — checks every 5 seconds with the same 3-failure threshold, temporarily removing a pod from Service routing (rather than restarting it) whenever it fails.

Together, these probes prevent downtime during rolling updates: new pods only receive traffic once genuinely ready, and pods that become internally stuck are automatically restarted without requiring manual intervention. The configuration was verified directly on the running pods and confirmed working as expected.

```
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl describe pod -l app=sampleapp | grep -A 3 "Liveness\|Readiness\|Startup"
  Liveness:  http-get http://:8080/health delay=0s timeout=1s period=10s successThreshold=1 failureThreshold=3
  Readiness:  http-get http://:8080/health delay=0s timeout=1s period=5s successThreshold=1 failureThreshold=3
  Startup:    http-get http://:8080/health delay=0s timeout=1s period=1s successThreshold=1 failureThreshold=30
  Environment: <none>
  Mounts:
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-p8xjm (ro)
  ...
  Liveness:  http-get http://:8080/health delay=0s timeout=1s period=10s successThreshold=1 failureThreshold=3
  Readiness:  http-get http://:8080/health delay=0s timeout=1s period=5s successThreshold=1 failureThreshold=3
  Startup:    http-get http://:8080/health delay=0s timeout=1s period=1s successThreshold=1 failureThreshold=30
  Environment: <none>
  Mounts:
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-brfjm (ro)
```

Figure 18: Liveness | Readiness | Startup kubectl check

8. Bonus Task — Horizontal Pod Autoscaler

Minikube's metrics-server addon was enabled as a prerequisite, providing the real-time per-pod CPU/Memory data the Horizontal Pod Autoscaler (HPA) needs to make scaling decisions. An HPA manifest was added, scaling the Deployment between 2 and 5 replicas, targeting 70% CPU and 80% Memory utilization relative to the resource requests configured in Task 4.

This was verified with a genuine, live load test rather than configuration alone: a temporary pod was created to continuously send requests to the application. CPU usage climbed from 1%

to 117% of the target, the HPA reacted by scaling from 2 to 4 replicas within seconds, and per-pod CPU usage normalized to roughly 50-60% as the same load was spread across more pods. After the load was stopped, the HPA gradually scaled back down from 4 to 3 to 2 replicas over several minutes, following its default stabilization window to avoid rapid, disruptive replica changes.

```
monitoring      prometheus-kube-prometheus-operator-754837c93f-entb5      1/1      Running      0      168m
monitoring      prometheus-kube-state-metrics-587c95c4c6-fcd7c      1/1      Running      0      168m
monitoring      prometheus-prometheus-kube-prometheus-prometheus-0      2/2      Running      0      168m
monitoring      prometheus-prometheus-node-exporter-skb8n      1/1      Running      0      168m
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ minikube addons enable metrics-server
🔦 metrics-server is an add-on maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
■ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
🌟 The 'metrics-server' add-on is enabled
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$
```

Figure 19: Enable metrics server add-on for minikube

```
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl apply -f k8s/hpa.yaml
horizontalpodautoscaler.autoscaling/sampleapp-hpa created
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get hpa
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: <unknown>/70%, memory: <unknown>/80%    2         5         0          15s
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl describe hpa sampleapp-hpa
Name:                sampleapp-hpa
Namespace:           default
Labels:              <none>
Annotations:         <none>
CreationTimestamp:   Tue, 01 Sep 2026 15:04:32 +0300
Reference:            Deployment/sampleapp-deployment
Metrics:              ( current / target )
  resource cpu on pods  (as a percentage of request):  1% (1m) / 70%
  resource memory on pods  (as a percentage of request):  34% (45394Ki) / 80%
Min replicas:        2
Max replicas:        5
Deployment pods:      2 current / 2 desired
Conditions:
  Type            Status  Reason                        Message
  ----            -
  AbleToScale     True    ScaleDownStabilized          recent recommendations were higher than current one, applying the highest recent recommendation
  ScalingActive   True    ValidMetricFound              the HPA was able to successfully calculate a replica count from cpu resource utilization (percentage of request)
  ScalingLimited  False   DesiredWithinRange            the desired count is within the acceptable range
Events:           <none>
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$

eog@eog-ThinkPad-X270-W10DG: ~
eog@eog-ThinkPad-X270-W10DG: ~ - 238x14
eog@eog-ThinkPad-X270-W10DG:~$ trux attach -t azure-local-agent
[detached (from session azure-local-agent)]
eog@eog-ThinkPad-X270-W10DG:~$ kubectl get pods -n kube-system | grep metrics-server
metrics-server-9d746b65-451af      0/1      Running      0      36s
eog@eog-ThinkPad-X270-W10DG:~$ kubectl top pods
NAME                                CPU(cores)   MEMORY(bytes)
sampleapp-deployment-6756487754-n5jg  1m           32Mi
sampleapp-deployment-6756487754-wpnt  1m           57Mi
eog@eog-ThinkPad-X270-W10DG:~$ kubectl top nodes
NAME                                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
minikube                            249m        6%       1833Mi          25%
eog@eog-ThinkPad-X270-W10DG:~$ kubectl delete pod load-generator
pod "load-generator" deleted from default namespace
eog@eog-ThinkPad-X270-W10DG:~$

eog@eog-ThinkPad-X270-W10DG: ~/GitHub/minikube-cicd 238x41
eog@eog-ThinkPad-X270-W10DG:~$ kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O http://sampleapp-service.default.svc.cluster.local/weatherforecast; done'
pod/load-generator created
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get hpa --watch
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 34%/80%    2         5         2          18m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 22%/70%, memory: 38%/80%    2         5         2          18m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 117%/70%, memory: 38%/80%    2         5         4          19m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 117%/70%, memory: 38%/80%    2         5         4          19m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 62%/70%, memory: 33%/80%    2         5         4          20m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 58%/70%, memory: 33%/80%    2         5         4          21m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 69%/70%, memory: 33%/80%    2         5         4          22m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 53%/70%, memory: 33%/80%    2         5         4          23m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 59%/70%, memory: 33%/80%    2         5         4          24m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 44%/70%, memory: 33%/80%    2         5         4          25m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 33%/80%    2         5         4          26m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 33%/80%    2         5         4          27m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 33%/80%    2         5         4          28m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 33%/80%    2         5         4          29m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 33%/80%    2         5         3          29m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 33%/80%    2         5         3          30m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 33%/80%    2         5         3          31m
sampleapp-hpa       Deployment/sampleapp-deployment  cpu: 1%/70%, memory: 36%/80%    2         5         2          31m
```

Figure 21: HPA | K8s cluster metrics with load generator (terminal)

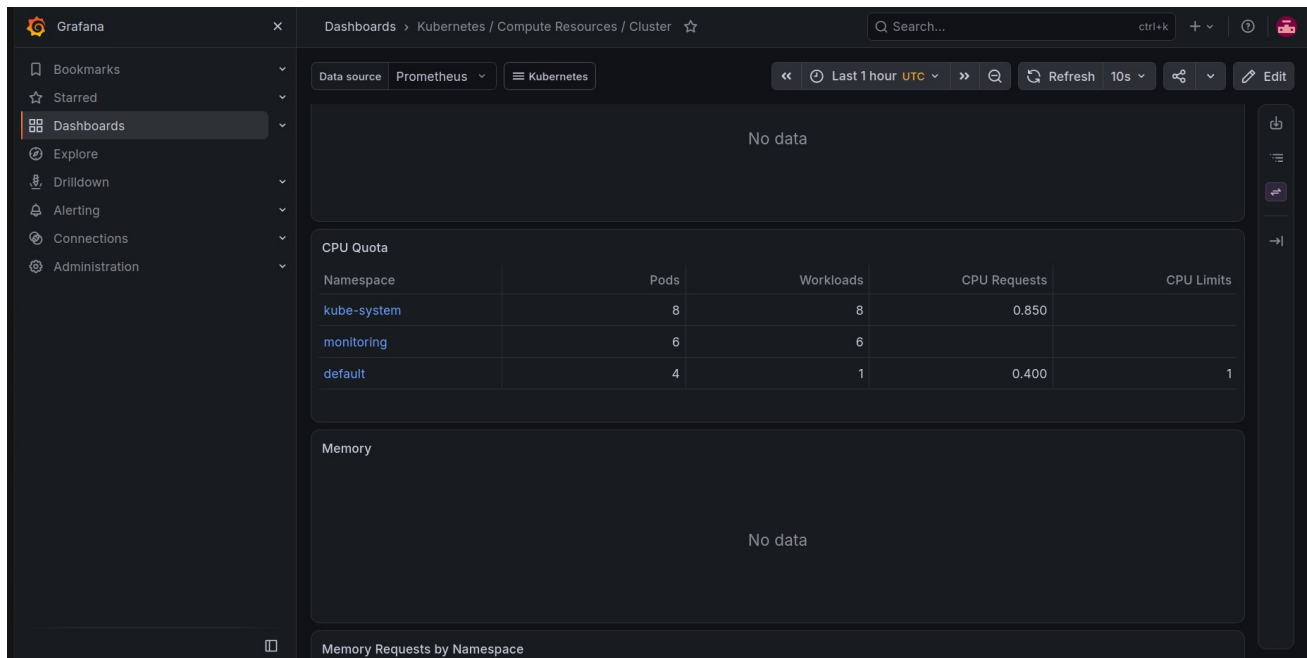


Figure 22: HPA | K8s cluster metrics with load generator (Grafana)

Horizontal scaling (adding more pods) was chosen deliberately over vertical scaling (making existing pods larger), for three reasons: vertical scaling requires restarting the container, causing brief downtime; it is capped by the physical resources of a single node; and it creates a single point of failure, whereas multiple smaller pods can be spread across different nodes in a real multi-node cluster.

9. Bonus Task — Automated Tests

A dedicated test project (SampleApp.Tests) was created using xUnit together with WebApplicationFactory, which spins up the real application in-memory and sends genuine HTTP requests against it, rather than testing mocked or isolated logic. Five tests were written, covering both application endpoints and validating that the Fahrenheit conversion logic is calculated correctly for every returned item.

Tests were integrated into the CI/CD pipeline at three distinct points, forming layered protection:

- Locally — run manually via dotnet test during development, for fast feedback.
- Pre-merge — a second, separate Azure Pipeline, sourced directly from GitHub via the official Azure Pipelines GitHub App, triggers automatically on every Pull Request. It runs a lighter Install-Restore-Build-Test sequence only, with no Docker build and no deployment, and reports its result back to GitHub as a native status check that blocks merging if it fails.
- Post-merge — the main CD pipeline re-runs the same test suite as part of its full build, blocking the Docker build, image push, and deployment stages entirely if any test fails.

A second, separate pipeline was required for the pre-merge check because a single Azure DevOps YAML pipeline is bound to one source repository; since Pull Requests are reviewed on GitHub while the main deployment pipeline is sourced from Azure Repos, reacting to GitHub Pull Request events required a pipeline whose source is GitHub itself.

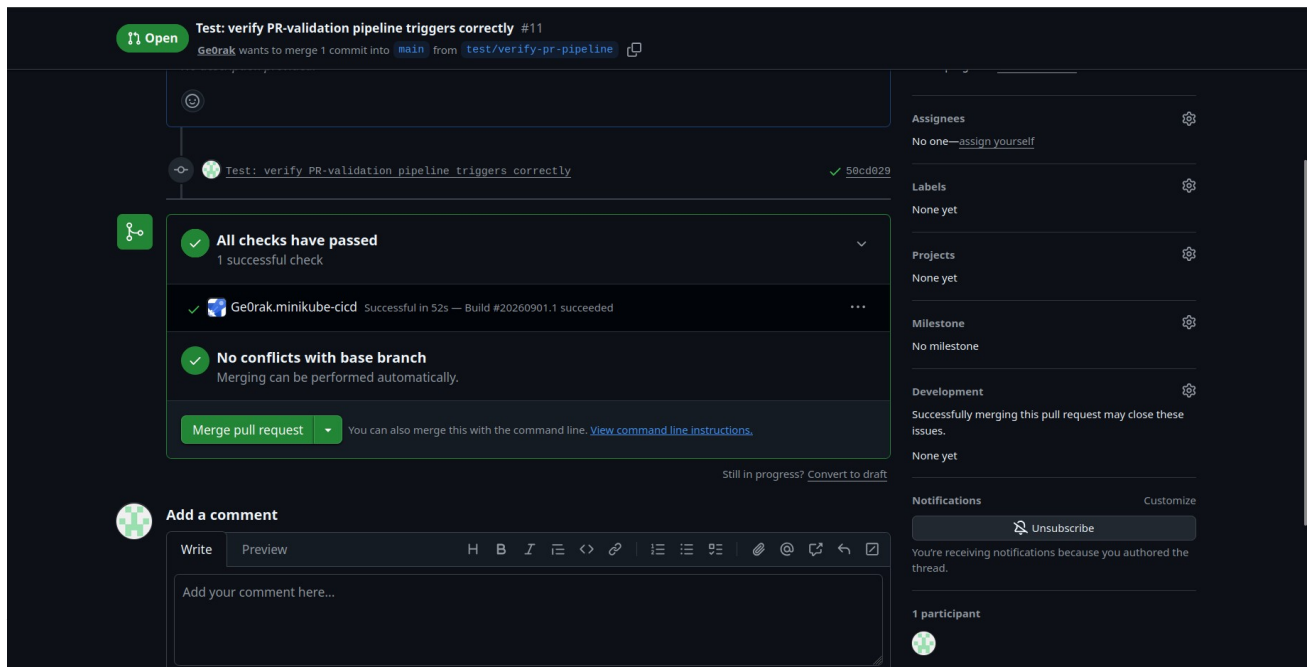


Figure 24: Pre-merge | Separate Azure Pipeline, sourced directly from GitHub

The screenshot displays the Azure DevOps interface for a pipeline named 'minikube-cicd'. The left sidebar shows the pipeline's structure, including jobs like 'Build and Push', 'Build', 'Run tests', and 'Deploy to Minikube'. The main panel shows the 'Run tests' task details, including a log of the test execution. The log shows the test suite 'SampleApp.Tests' being run successfully, with a duration of 1s and 5 tests passed.

Figure 24: Post-merge | Main CD pipeline re-runs the same test suite as part of its full build

10. Bonus Task — Multi-Region / Multi-Zone Failover Strategy

This final bonus item is a conceptual architectural design rather than an implementation, since a genuine multi-region deployment requires real infrastructure across multiple cloud regions, outside the scope of a local Minikube-based assessment.

Multi-zone resilience

Within a single region, nodes and replicas would be spread across multiple Availability Zones using topology spread constraints, so that the failure of one physical data center does not take down every replica. **The Readiness probe already implemented in this project is exactly the mechanism that makes automatic rerouting away from a failed zone possible.**

Multi-Zone Redundancy within a Region

Availability-zone isolation keeps pods running if one data center fails

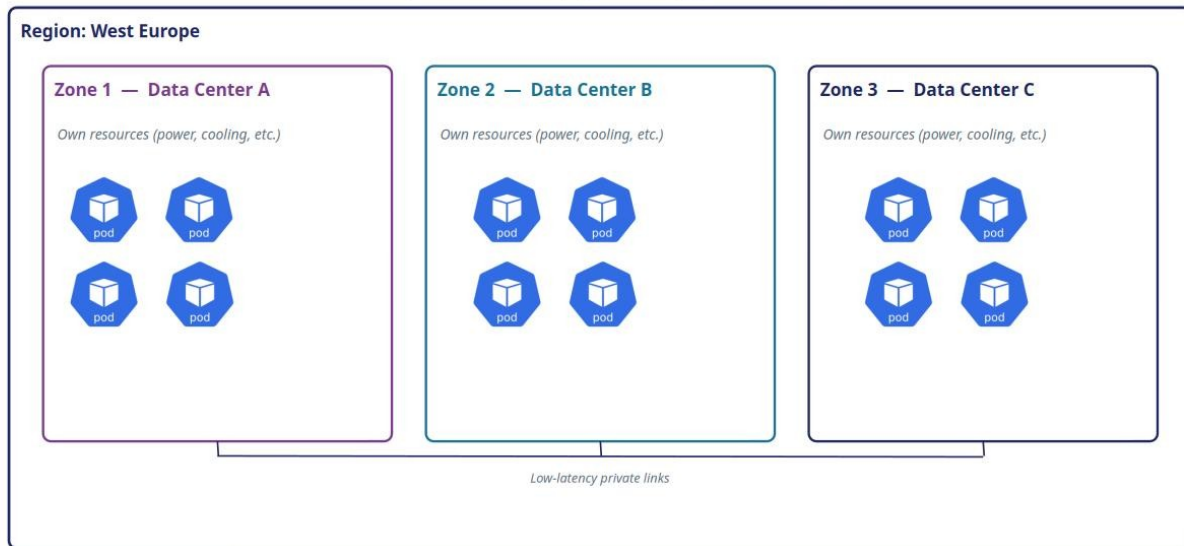


Figure 25: Multi-zone resilience topology

Multi-region resilience

Beyond a single region, independent, self-sufficient Kubernetes clusters would run in two or more geographic regions, each deployed by an extended version of the same CI/CD pipeline already built. A Global Load Balancer — such as Azure Front Door, Cloudflare, or AWS Global Accelerator — would continuously health-check each region's entry point and redirect traffic away from a region experiencing failures, automatically and without manual DNS changes.

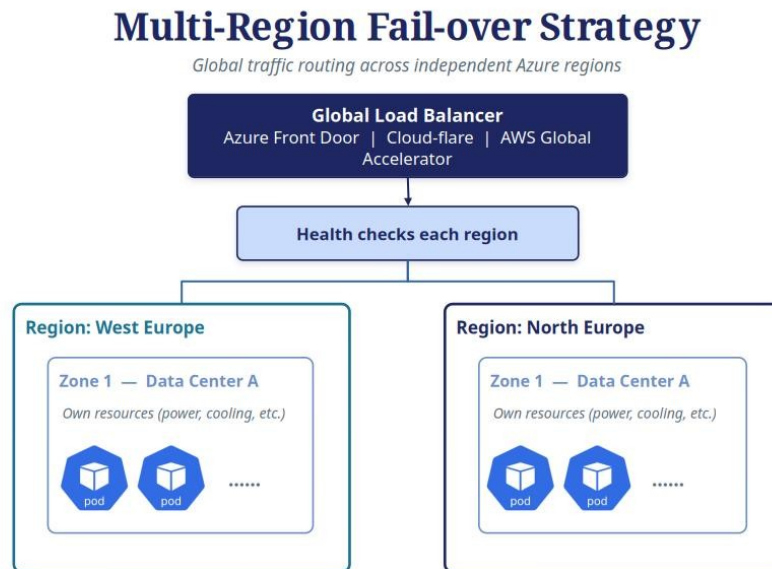


Figure 26: Multi-region resilience topology

Data replication

The sample application in this project is stateless, so there is no data replication gap to address. For a hypothetical future version with persistent storage, an Active-Passive replication strategy is recommended as the pragmatic starting point: one region holds the primary, writable database while others maintain continuously-updated read replicas. This means you might lose a few seconds of the most recent writes if a region fails — a small, acceptable trade-off for keeping the system much simpler than Active-Active, which would require handling conflicts when the same data is written in two regions at once.

11. Challenges Faced

Several genuine obstacles were encountered and resolved during this project, each contributing to a better final result:

- Ubuntu 26.04 was too new for some tools. Docker's official repo didn't support it yet, so we pointed it at Ubuntu 24.04's repo instead — which works fine. We also used .NET 10 (available via apt) instead of .NET 8.
- Docker copies your whole project folder into the image — including your local bin/ and obj/ folders, which hold leftover build info from your own machine. That leftover info clashed with the fresh install happening inside the container, causing a confusing

"package not found" error, even though the package was fine. Adding a .dockerignore file to skip bin/ and obj/ fixed it.

- The self-hosted agent's first registration attempt failed with an authorization error, resolved by regenerating the Personal Access Token with broader scope.
- Azure's cloud pipeline agents can't reach a Kubernetes cluster running only on your laptop. The real solution was installing a self-hosted agent on the laptop itself, so the deploy step runs locally where the cluster actually is.
- Many .NET apps like this one hide their Program class and by default it's private to the project — invisible to the separate test project. Since the test project needs to reference it directly to spin up the real app for testing, one line (public partial class Program {}) was added to make it visible. It doesn't change how the app runs at all — it's a widely-used, standard fix for exactly this situation.